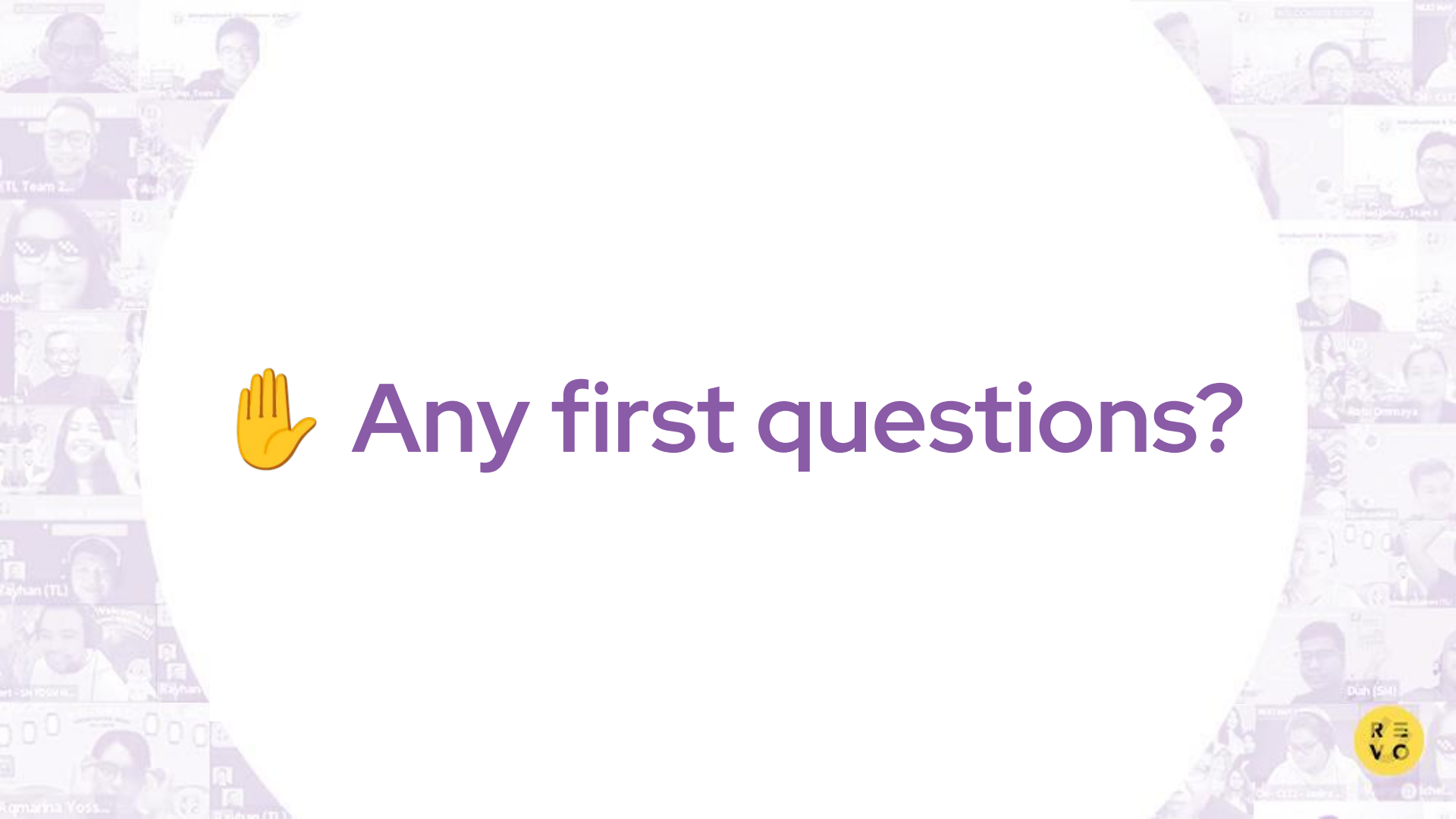
Software Engineering

Dependency Injection

What we will learn...

- Dependency Injection Fundamentals
- Providers in Nest.js
- Provider Registration and Module System


✋ Any first questions?

What is Dependency Injection?

What is Dependency Injection?

Dependency Injection (DI) is a design pattern where a class receives its dependencies from external sources rather than creating them itself. This pattern shifts responsibility for creating and managing dependencies to a container (in this case, NestJS).

Without DI

(AKA: "class yang insecure dan pengen ngapa-ngapain sendiri")

```
export class UserService {
  private readonly repo: UserRepository;

  constructor() {
    // Tight coupling hell
    this.repo = new UserRepository();
  }

  findAll() {
    return this.repo.getUsers();
  }
}
```

Why this is a disaster:

- Hard to test
- Hard to replace dependencies
- Hard to mock
- Hard to maintain
  Basically like when someone mixes kecap manis and sambal but refuses to measure anything because "feeling aja".

What is Dependency Injection?

With DI

(AKA: “class yang akhirnya belajar delegasi kayak orang normal”)

```
@Injectable()
export class UserService {
  constructor(private readonly repo:
UserRepository) {}

  findAll() {
    return this.repo.getUsers();
  }
}
```

What magically gets better:

- No more manual `new UserRepository()`
- Dependencies can be mocked, swapped, extended
- Your tests stop crying
- Your code stops behaving like Singapore MRT peak hour chaos

Benefits of Dependency Injection

- **Loose Coupling**
  Your components stop clinging to each other like teenagers in a Bintan mall. They depend on abstractions, not hardcoded classes.
- **Enhanced Testability**
  Mocking dependencies becomes easy. Your unit tests finally behave like responsible adults instead of breaking everything.
- **Improved Maintainability**
  You can replace implementations without touching the classes that use them. Feels a bit like upgrading kopi sachets to proper beans without rebuilding your whole kitchen.
- **Component Reusability**
  Services can be reused anywhere in your app. One clean provider, many happy modules.

Good DI is like having a proper mechanic. Bad DI is when you DIY everything, lose a bolt, and suddenly your motor beat sounds like dangdut at 5am.

What are Providers?

Providers are the backbone of DI in NestJS. They're basically the grownups in your app, holding all the business logic while controllers sit there acting like they're the main character. Anything **decorated** with `@Injectable()` can be **injected elsewhere**, which is Nest's fancy way of saying "please stop manually wiring everything like it's 2009."

```
import { Injectable } from '@nestjs/common';

@Injectable()
export class ProductsService {
  findAll() {
    return ['mie goreng', 'teh botol', 'roti kaya'];
  }

  create(product: string) {
    return `Product created: ${product}`;
  }
}
```

What are Providers?

Injecting providers in NestJS is basically **constructor** VIP access. You declare what you want, Nest hands it to you, and nobody has to pretend they enjoy writing `new SomethingService()` everywhere.

```
import { Controller, Get } from '@nestjs/common';
import { ProductsService } from './products.service';

@Controller('products')
export class ProductsController {
  constructor(private readonly productsService: ProductsService) {}

  @Get()
  findAll() {
    return this.productsService.findAll();
  }
}
```

Custom Providers

useClass

`useClass` lets you swap out one implementation for another. Great when you want different behaviours for development, production, or when your senior dev says *“we’ll refactor later”* but it's 2027 and that code is still there.

```
providers: [
  {
    provide: PaymentService,
    useClass: process.env.NODE_ENV === 'production'
      ? StripePaymentService
      : MockPaymentService,
  },
],
```

- `provide`: The token you inject elsewhere.
- `useClass`: The concrete class Nest will instantiate and inject.
- You can conditionally choose implementations because apparently life wasn’t chaotic enough.

Custom Providers

useValue

The `useValue` syntax lets you provide constant values or simple objects instead of a full class-based provider. This is useful when you need to inject configuration, mocks, or static data.

```
// config.constants.ts
export const APP_CONFIG = {
  appName: 'My Awesome App',
  version: '1.0.0',
};
```

```
  providers: [
    {
      provide: 'CONFIG',
      useValue: APP_CONFIG,
    },
  ],
```

```
import { Inject, Injectable } from
'@nestjs/common';

@Injectable()
export class AppService {
  constructor(@Inject('CONFIG') private config)
{}

  getInfo() {
    return `${this.config.appName}
v${this.config.version}`;
  }
}
```

Custom Providers

useFactory

The `useFactory` syntax creates providers dynamically. Instead of giving Nest a class or a constant value, you give it a function that *returns* the value. This is perfect when the value depends on some runtime logic, environment variables, async operations, or other providers.

Factories can also depend on other providers. Declare them using the `inject` array.

```
// example.service.ts
import { Injectable } from '@nestjs/common';

@Injectable()
export class ConfigService {
  getDbHost() {
    return 'localhost';
  }
}
```

```
  providers: [
    ConfigService,
    {
      provide: 'DB_CONNECTION',
      useFactory: (config: ConfigService) => {
        const host = config.getDbHost();
        return `Connected to database at
${host}`;
      },
      inject: [ConfigService],
    },
  ],
```

Custom Providers

useFactory

This pattern shines when you need more control over how a provider is created.

You'd use `useFactory` when:

1. **Provider creation requires logic**
   Maybe you need to compute something, generate a token, read environment variables or decide between two values. A factory function lets you do all of that before returning the final provider value.
2. **Provider depends on other providers**
   Factories can receive other services as parameters. This makes it easy to build a provider using information from another service, such as a config loader, a database connection builder, or anything that isn't just a static class.
3. **You need to create providers conditionally**
   If your app needs to decide which implementation to use at runtime. For example, choosing between a mock service and a real service, picking configs based on environment, or generating objects only under certain conditions.

Provider Registration and Module System

Exporting Providers

In NestJS, a provider only lives inside the module where it is registered. If another module wants to use that provider, you must **export** it. Exporting basically means:
“Oi module next door, you’re allowed to use this thing.”

```
import { Injectable } from '@nestjs/common';

@Injectable()
export class UserService {
  findAll() {
    return ['andi', 'budi', 'ciko'];
  }
}
```

```
// user.module.ts
import { Module } from '@nestjs/common';
import { UserService } from './user.service';

@Module({
  providers: [UserService],
  exports: [UserService]  // expose it to other
modules
})
export class UserModule {}
```

Provider Registration and Module System

Global Providers

Sometimes you want a provider to be available **everywhere** without having to import its module in every other module. NestJS gives you a clean way to do that:
make the module **global**.

This is perfect for things like configuration services, logging services, shared utilities, database connections, and any other thing you secretly copy paste across projects.

You do this with the `@Global()` decorator.

```
// common.module.ts
import { Module, Global } from '@nestjs/common';
import { ConfigService } from './config.service';

@Global()
@Module({
  providers: [ConfigService],
  exports: [ConfigService]
})
export class CommonModule {}
```

Common Module

A **Common Module** in NestJS is a dedicated module that stores **reusable, shared functionality** used across multiple other modules in your application. Instead of copy pasting services or importing the same providers repeatedly like some weird ritual, you put all cross-cutting logic in one place and let the rest of your application reuse it cleanly.

The Common Module is not tied to any specific business domain. It exists only to house general-purpose utilities such as:

- Shared services like LoggerService, ConfigService or PrismaService (I promise we will teach these in 2-3 weeks)
- Shared pipes, guards and interceptors (also these)
- Helper functions and decorators (maybe these)
- Anything that multiple modules depend on but doesn’t belong to a specific feature (surprise!)

Mini Quiz

Request Processing Pipeline with NestJS Middleware

Basic Concepts

1. What is the main purpose of Dependency Injection in NestJS?

   a) To automatically generate database models

   b) To make components loosely coupled and more testable

   c) To create API documentation

   d) To improve application performance

1. In NestJS, which decorator is used to mark a class as injectable?

   a) @Service()

   b) @Component()

   c) @Injectable()

   d) @Provider()

1. The principle where control is transferred from your code to the framework is called:

   a) Control Reversal

   b) Inversion of Control

   c) Dependency Reversal

   d) Framework Control

Request Processing Pipeline with NestJS Middleware

Provider Types

4. Which syntax allows you to provide a constant value instead of a class in NestJS?

   a) useValue b) useConstant

   c) provideValue d) injectValue

4. When you want to dynamically create a provider based on certain conditions, which provider type should you use?

   a) useClass b) useFactory

   c) useDynamic d) useConditional

Request Processing Pipeline with NestJS Middleware

Practical Application

6. Which is the correct way to inject a provider in a NestJS controller?

a) constructor(@Inject() private service: MyService) {}

b) constructor(private readonly service: MyService) {}

c) constructor: new MyService()

d) @Inject(MyService) service;

Thank you!

The only way to become a great Software Engineer is to get your hands dirty in the code. Break things, debug relentlessly, and learn why it works (or crashes).

🚀 Code. Break. Refactor. Repeat.

Share your learning journey with us!
Let's inspire each other with your stories

@revou_id RevoU revoudotco